

# Project: Saylani Student Ops Desk

## A spec-driven build — requirements only, two hours

This is a specification, not a tutorial. There are no prompts to copy. You will write a spec, hand it to your coding agent — Claude Code or OpenCode — and be responsible for everything it produces.

**The brief.** The Ops Desk is the front door for student questions about this bootcamp. A student asks something in plain language; the Desk works out whether it is about an assignment, a career question, or course administration, answers it from real course data, and closes every conversation with a structured ticket your systems could file. It refuses anything that isn't about the course, it knows who is asking without being told in the prompt, and every run it performs can be audited after the fact.

Everything you need is in the fundamentals guide. This project uses all of it.

## The rules

1. **Specification before implementation.** No source file is written until Phase 0 is complete and committed. Your git history is the evidence, and it is graded.
2. **The agent writes the code. You own it.** Anything you cannot explain, you have not finished. The viva at the end assumes you read every diff.
3. **Requirements are numbered and testable.** "FR-7 works" means you can demonstrate it on demand, not that the code looks plausible.
4. **The clock is real.** When you fall behind, cut from the bottom of the cut list — not from whatever is hardest.

## The clock

| Time      | Phase                              | What exists at the end of it                  |
|-----------|------------------------------------|-----------------------------------------------|
| 0:00–0:35 | Phase 0 — specify                  | Four spec artifacts, committed, no code       |
| 0:35–1:05 | Phase 1 — core desk                | FR-1 ... FR-4 running in the terminal         |
| 1:05–1:35 | Phase 2 — specialists              | FR-5 ... FR-9                                 |
| 1:35–1:55 | Phase 3 — operations and interface | FR-10 ... FR-13                               |
| 1:55–2:00 | Demo                               | One clean conversation, one trace, one ticket |

**Cut list, in this order:** FR-11, then FR-6, then the **AgentHooks** half of FR-10. Never cut FR-8 or FR-7 — the guardrail and the ticket are what the viva is built on.

## Phase 0 — the specification gate

---

Produce four documents before any implementation. Spec Kit generates this shape for you; plain markdown is acceptable as long as the four exist and say the same things.

**constitution.md** — the rules the build may not violate. At minimum: which model provider is used and at which level it is configured; that secrets live only in `.env`; that tools never raise to the caller; that no student data reaches the model except through instructions you wrote deliberately.

**spec.md** — what the Desk does, in behaviour, not implementation. Every requirement below restated in your own words, plus the three things this project explicitly will *not* do.

**plan.md** — the architecture: which agents exist, which owns the conversation, what each tool is called and what it returns, and the shape of every data structure crossing a boundary.

**tasks.md** — an ordered list of implementation tasks, each small enough to verify on its own, each naming the requirement it satisfies.

**Gate:** all four committed before the first line of code. A single commit containing both a spec and an implementation fails this phase regardless of whether the code works.

---

## Phase 1 — the core desk

---

### FR-1 — A Gemini-backed agent, configured at the agent level

The Desk runs on `gemini-2.5-flash` through an OpenAI-compatible client, with the model set on the agent itself rather than globally or per run. The entry point is asynchronous.

**Done when:** a question typed in the terminal is answered by Gemini; no `set_default_openai_client` appears anywhere; the program's entry point is an `async` function driven by `asyncio.run`.

## FR-2 — Course knowledge, reachable only through tools

Course facts live in a `courses.json` file you write, and the agent can only reach them by calling tools. At minimum the Desk can list courses, fetch one course's schedule and policies, and look up an assignment by id.

```
{
  "courses": [
    {
      "id": "agentic-ai-w4",
      "title": "Agentic AI - weekdays batch 4",
      "schedule": "Mon-Thu, 7-9pm",
      "policies": {"late_submission": "48 hours, 20% penalty"},
      "assignments": [
        {"id": "a3", "title": "First coded agent", "due": "2026-10-02"}
      ]
    }
  ]
}
```

**Done when:** deleting a course from the file removes it from the Desk's answers with no code change, and the Desk declines to invent an assignment id that is not in the file.

## FR-3 — The student is in context, never in the prompt

A `StudentProfile` is passed to every run as local context. Tools read it. The prompt text never contains the student's name, roll number or tier.

```
from dataclasses import dataclass

@dataclass
class StudentProfile:
    name: str
    roll_no: str
    course_id: str
    tier: str = "regular"          # "regular" or "scholarship"
    open_tickets: int = 0
```

**Done when:** the generated schema for a profile-reading tool contains no wrapper parameter, and grepping your source for the student's name finds it only in the object you constructed.

## FR-4 — Instructions that change per turn

The Desk's system prompt is built at request time from the profile: it greets the student by name, names the course they are enrolled in, and becomes terser once `open_tickets` is 3 or more.

**Done when:** three different profiles produce three visibly different system prompts, and you can print the resolved prompt before any model call happens.

---

## Phase 2 — the specialists

---

### FR-5 — Two specialists, cloned from one base, reached by handoff

An **Assignments** specialist and a **Careers** specialist are produced by cloning a single base agent, differing only in instructions and model settings. The Desk transfers the conversation to whichever one fits the question; the specialist, not the Desk, answers the student.

Assignments runs cold and factual. Careers runs warmer. Both settings must be deliberate and defensible.

**Done when:** the agent that answered is identifiable in code after the run, the handoff appears in the run's items, and the specialists share the base agent's model without restating it.

### FR-6 — One specialist exposed as a tool, not a handoff

A **Summariser** condenses a long policy answer to three lines. It is wired as a tool the Desk calls, so the Desk keeps the conversation and speaks in its own voice.

**Done when:** you can state why this one is a tool while the other two are handoffs, and the final message after a summarisation still comes from the Desk.

### FR-7 — Every resolved conversation produces a structured ticket

The Desk's final output for a resolved query is a typed object, not prose.

```
from typing import Literal
from pydantic import BaseModel

class Ticket(BaseModel):
    category: Literal["assignment", "career", "admin"]
    summary: str
    next_step: str
    resolved: bool
    escalate: bool
```

**Done when:** `type(result.final_output)` is `Ticket`, `resolved` is used in a Python `if` rather than read by a human, and a deliberately impossible request surfaces the SDK's parsing error instead of a half-filled object.

## FR-8 — A guardrail that refuses non-course questions

An input guardrail rejects anything unrelated to the bootcamp before the Desk's model runs. The program catches the tripwire and replies politely; it must not crash.

**Done when:** an off-topic question produces a courteous refusal, the refusal costs nothing at the model that would have answered it, and you can point to where the exception was caught.

## FR-9 — Tool gating, a stopping rule, and a ceiling

Three separate controls, all present:

- A tool only offered to students whose **tier** is **scholarship** — absent, not refused, for everyone else.
- A **close\_ticket** tool that ends the run the moment it is called, its output becoming the final result.
- A turn ceiling that raises rather than loops, caught and reported.

**Done when:** the same question run as **regular** and as **scholarship** offers the model a different set of tools, and you can name the number you chose for the ceiling and why.

---

# Phase 3 — operations and interface

---

## FR-10 — An audit trail across the whole run, and one agent watched closely

Run-level hooks record an ordered timeline covering every agent in a conversation, including the handoff. Separately, agent-level hooks are attached to exactly one specialist.

**Done when:** one student question yields one timeline that names both agents in order, and you can explain why the agent-level hooks go quiet at the moment the handoff happens.

## FR-11 — A custom runner wrapping every run

A custom runner stamps a request id and elapsed time around every run in the process, registered once at startup. No agent definition changes to accommodate it.

**Done when:** the wrapper's output appears for the Desk's run and for the specialist's run, and no agent file mentions it.

## FR-12 — A Chainlit interface with per-session memory

The Desk is usable in a browser. The agent and the student's profile are built once when the session opens, not per message. The conversation remembers earlier turns.

**Done when:** a second message refers to the first and is understood, two browser windows do not share history, and the handler awaits the run rather than calling the synchronous variant.

## FR-13 — Traceable conversations

Tracing is on, exported under your own key, and a single student conversation appears as one trace rather than several.

**Done when:** you can open the trace, name every span in it, and point to one call the Desk made that it did not need to make.

---

## Non-functional requirements

---

- **NFR-1 — Secrets.** Keys live in `.env`, which is gitignored. A missing key produces a clear startup error, not a stack trace three layers deep.
- **NFR-2 — Cost.** Every agent declares its own model settings. Nothing generates without a ceiling.
- **NFR-3 — Observability.** Every conversation is traceable, and the audit timeline from FR-10 is written somewhere durable, not only printed.
- **NFR-4 — Failure.** A tool that hits bad data returns a sentence the model can act on. A tool that raises into the runner is a defect, not a feature.
- **NFR-5 — Provenance.** `git log` shows the four Phase 0 artifacts committed before the first code commit. This is checked.

## Concept coverage

---

Every part of the fundamentals guide is forced by a requirement above. If you cut something not on the cut list, this table tells you what you lost.

| Part                               | Forced by                               |
|------------------------------------|-----------------------------------------|
| 0–2 Setup, keys, Gemini            | FR-1, NFR-1                             |
| 3 <b>Runner</b> and <b>asyncio</b> | FR-1, FR-12                             |
| 4 Model configuration              | FR-1                                    |
| 5 Tools                            | FR-2                                    |
| 6 Model settings                   | FR-5, NFR-2                             |
| 7 Local context                    | FR-3                                    |
| 8 Dynamic instructions             | FR-4                                    |
| 9 Cloning                          | FR-5                                    |
| 10 Tracing                         | FR-13                                   |
| 11 Agents as tools                 | FR-6                                    |
| 12 Handoffs                        | FR-5                                    |
| 13 Advanced tool control           | FR-9, NFR-4                             |
| 14 Structured output               | FR-7                                    |
| 15 Guardrails                      | FR-8                                    |
| 16 Lifecycle hooks                 | FR-10                                   |
| 17 Run lifecycle hooks             | FR-10                                   |
| 18 Custom runners                  | FR-11                                   |
| 19 Chainlit                        | FR-12                                   |
| 20 Practice with an agent CLI      | The whole project — you are driving one |

## Definition of done

---

| Requirement                                 | How it is checked                                                              |
|---------------------------------------------|--------------------------------------------------------------------------------|
| Spec preceded code                          | <code>git log</code> order, Phase 0 artifacts first                            |
| Desk answers from <code>courses.json</code> | Delete a course, answer changes                                                |
| Profile never in the prompt                 | Tool schema has no wrapper; grep the source                                    |
| Prompt changes per student                  | Three profiles, three resolved prompts                                         |
| Handoff reaches a specialist                | Answering agent identified after the run                                       |
| Ticket is typed                             | <code>type(final_output)</code> is <code>Ticket</code> , branched on in Python |
| Off-topic is refused, cheaply               | Refusal shown, no billed call at the Desk model                                |
| Tools differ by tier                        | Same question, two tiers, two tool sets                                        |
| One conversation, one trace                 | Trace opened and every span named                                              |
| Runs in a browser with memory               | Second message understood, windows isolated                                    |

## Defending your work

---

Eight questions. A student who let the agent think for them cannot answer these.

1. Show the schema for a tool that reads the profile. Why is the wrapper parameter missing from it?
2. A blocked question cost you nothing. Prove it, from the trace.
3. Which attributes do your two specialists share with the base agent, and which are their own?
4. Rename one specialist. What silently degrades, and where does that name come from?
5. Why does the Chainlit handler await the run? What is the exact error if it does not?
6. Your ticket came back missing a field. Which exception, raised by which layer?
7. Which hook fires once per agent and which fires once per model call? Give the counts for one ticket.
8. What does your custom runner see that your hooks cannot?

## If you finish early

---

- Give the handoff a typed input so the Desk must state *why* it is transferring.
- Persist tickets to SQLite and let the Desk answer "what did I ask last week?".
- Add a second guardrail on the output that refuses any answer quoting a policy not in `courses.json`.
- Replace `courses.json` with your own class's real data and find out what breaks.